DAA Assignment 2

NAME:ALURI POOJITHA
REG NO:192511230

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Brute-Force and Exhaustive-Search techniques – Selection Sort, Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Q1 – Sorting Under Different Data Conditions **6 Marks**

Problem Context – E-Commerce Order Processing

An e-commerce fulfilment centre arranges customer orders according to a **dispatch priority score**.

The development team is considering:

Selection Sort | Bubble Sort | Insertion Sort

However, the incoming data may be randomly ordered, nearly sorted, already sorted or significantly disordered.

Your Challenge

Investigate whether one of the three algorithms should always be preferred or whether the appropriate choice depends on the **characteristics of the input data**.

Before implementation:

- Predict how the three algorithms will behave under different input arrangements.
- Explain the algorithmic reason for your prediction.

Design **two deliberately contrasting datasets** and **one additional edge-case dataset of your own** that will expose meaningful behavioural differences among the algorithms.

Implement the three algorithms and record appropriate measures such as:

- Number of comparisons
- Number of swaps/movements
- Execution time, where useful

Compare your prediction with the observed behaviour.

Design Decision

The fulfilment centre reports that approximately **95% of existing orders are already in the correct priority order**, with only small batches of new orders added periodically.

Determine which of the three algorithms you would recommend and defend your decision.

Evidence Expected

Reasoned prediction, three deliberately designed input conditions, implementation of the three sorting algorithms, operation counts, comparison of predicted and observed behaviour, and an evidence-based recommendation.

Q2 – When Is Sequential Search an Appropriate Engineering Choice? 4 Marks

Problem Context – Aircraft Maintenance Tool Tracking

An aircraft maintenance facility maintains a temporary list of specialised tools currently issued to technicians.

Each record contains:

Tool ID | Technician ID | Issue Time | Work Bay

The list changes continuously as tools are issued and returned.

When a technician asks:

> "Who currently has Tool T-184?"

the system uses **Sequential Search**.

Your Challenge

Design experiments to investigate Sequential Search under:

- Successful searches at different positions

- Unsuccessful searches
- Different list sizes

Record the **number of comparisons** required.

Then answer:

> **Under what operating conditions is Sequential Search a reasonable engineering choice, and when would you stop recommending it?**

Consider both:

- Dataset size
- Frequency of search requests

Changed Condition

The organisation expands from one hangar to **20 maintenance facilities**, resulting in substantially more active records and lookup requests.

Re-evaluate your original recommendation.

Evidence Expected

Designed search experiments, comparison-count evidence, analysis of successful and unsuccessful searches, an evidence-based operating boundary and a justified response to the changed operating condition.

Q3 – Engineer the Behaviour of Brute-Force String Matching 6 Marks

Problem Context – Cybersecurity Log Screening

A security operations centre searches large text-based system logs for known event signatures represented as character patterns.

The initial system uses **Brute-Force String Matching**.

Your Challenge

Instead of merely implementing the algorithm, investigate **how the structure of the text and pattern influences computational effort**.

Design:

1. One text-pattern pair in which the algorithm performs relatively little work.

2. One text-pattern pair deliberately engineered to cause a **large number of character comparisons**.
3. One additional test condition of your own choice.

Before executing each test:

- Predict the expected behaviour.
- Explain the algorithmic reason for your prediction.

Implement the algorithm and record the number of character comparisons.

Inquiry

Compare your prediction with the experimental result.

Explain:

- Why one input requires substantially more work than another
- Which characteristics of the text and pattern create repeated comparisons
- What this reveals about evaluating an algorithm using only its input size

Evidence Expected

Correct implementation, three deliberately designed test conditions, prediction before execution, character-comparison evidence, and a reasoned explanation of the observed differences.

Q4 – Closest Pair and the Cost of Examining Every Possibility 6 Marks

Problem Context – Drone Collision-Risk Monitoring

A logistics company operates delivery drones. At a particular instant, each drone reports its position:

[
D_i=(x_i,y_i)
]

The monitoring system must identify the **two drones closest to one another**.

Your Challenge

Design a **Brute-Force Closest-Pair** solution.

Before implementation, determine:

- Which candidate pairs must be examined
- How duplicate pair comparisons can be avoided
- How many unique pairs exist for (n) drones

Derive the number of candidate pairs mathematically.

Implement your solution and test it using **at least three meaningfully different input sizes**.

Record:

- Number of pairs examined
- Number of distance calculations
- Execution time, where useful

Compare the theoretical growth with experimental behaviour.

Scalability Inquiry

Define what you consider an **acceptable response time** for your experimental environment.

Using your evidence, determine approximately when the brute-force approach begins to become impractical.

Finally, identify **where the computational inefficiency originates** and describe what a redesigned strategy would need to avoid or reduce.

You are not required to implement an advanced Closest-Pair algorithm in this question.

Evidence Expected

Algorithm design, mathematical pair-count derivation, implementation, experimental growth evidence, a defined acceptability criterion and reasoned identification of the limitation of the brute-force strategy.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle 7 Marks

Problem Context – Wildlife Reserve Boundary Monitoring

Monitoring stations are positioned throughout a wildlife reserve. Each station is represented by coordinates:

[
P_i=(x_i,y_i)
]

Conservation officers need to determine the **outer boundary formed by the monitoring stations**.

You are required to develop a **Brute-Force Convex Hull** solution.

Design Starting Point

You are given only the following geometric principle:

> **A line joining two candidate points may form part of the outer boundary if all remaining points lie on an appropriate single side of that line.**

You are **not provided with the complete algorithm**.

Your Challenge

Convert this geometric property into a computational procedure.

Determine:

- How two candidate points define a line
- How the position of another point relative to that line can be determined
- How the remaining points must be examined
- What condition identifies a potential hull edge
- How the final boundary will be constructed

Develop your own pseudocode before implementation.

Test Your Design

Test the algorithm using:

- A point set containing both boundary and interior points
- A second point configuration deliberately designed by you to challenge your algorithm

Visualise the resulting boundary.

Analyse how the computational work grows as the number of points increases.

Correctness Inquiry

Explain:

> **Why does your geometric test provide evidence that an accepted edge belongs to the outer boundary?**

Your explanation should be based on geometric reasoning rather than simply referring to the program output.

Evidence Expected

A student-derived geometric decision rule, pseudocode, implementation, deliberately designed test configurations, computational analysis, boundary visualisation and a reasoned correctness argument.

Q6 – Exhaustive Search and Combinatorial Growth **8 Marks**

Problem Context – Humanitarian Airlift Planning

A helicopter carrying emergency supplies has a strict payload limit.

Each available package contains:

Package ID | Weight | Humanitarian Value

The objective is to select a combination of packages that:

- Does not exceed the payload limit, and
- Maximises the total humanitarian value delivered.

Your Challenge

Design an **Exhaustive-Search strategy**.

Determine:

- How candidate package combinations will be represented
- How candidate solutions will be generated
- How feasibility will be checked
- How feasible solutions will be evaluated
- How the best solution will be retained

Do not begin from supplied pseudocode.

Correctness Check

Create a small instance for which the possible combinations can be independently verified.

Use this to establish whether your algorithm identifies the correct optimum.

Growth Investigation

Investigate increasing problem sizes.

Before experimentation:

- Derive/predict how the number of candidate combinations grows with (n).
- Predict what will happen as additional packages are introduced.

Use **at least three meaningful problem sizes** to validate your prediction experimentally.

Inquiry

Explain:

> **Why can adding what appears to be a relatively small number of additional packages produce a dramatic increase in computational effort?**

Your explanation must relate to the growth of the **solution space**.

Deployment Decision

Assume that the airlift planner must produce a loading decision within a fixed response time selected and justified by you.

Using your experimental evidence, estimate the largest problem size for which you would be willing to deploy your exhaustive-search solution.

Evidence Expected

Student-designed exhaustive enumeration, correctness verification, mathematical prediction of solution-space growth, experimental validation, explanation of combinatorial growth and an evidence-based deployment boundary.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate **13 Marks**

Review the evidence you generated while working on Q1–Q6.

Select **ONE solution** that demonstrates a significant computational limitation as its problem size or operating requirement increases.

You are **not permitted simply to replace the solution with a more advanced textbook algorithm and state that its complexity is better**.

Your task is to understand the computational problem first and then develop an appropriate redesign.

A. Diagnose the Computational Bottleneck **2 Marks**

Using your own mathematical and/or experimental evidence, identify:

- Where most computational effort is being spent
- Which operation grows significantly as the input increases
- Whether work is being unnecessarily repeated
- Which characteristic of the original algorithm causes the limitation

Evidence Expected

A precise diagnosis supported by evidence from your original implementation and analysis.

B. Define the Design Objective **2 Marks**

Before proposing a new solution, clearly state what your redesign must achieve.

For example:

- Reduce unnecessary comparisons
- Avoid examining every candidate
- Exploit existing ordering
- Reuse previously computed information
- Reduce the number of candidate solutions
- Reduce repeated processing

Specify a **measurable objective** wherever possible.

Evidence Expected

A clearly stated redesign objective derived directly from the identified computational limitation.

C. Design the Revised Computational Strategy **4 Marks**

Develop a revised solution.

Explain:

- What you have changed
- What information or property your new design exploits
- Which operations are avoided or reduced
- Whether preprocessing is introduced
- Whether additional memory is required
- Any new trade-offs created by the redesign

Provide appropriate pseudocode or a design representation.

Your marks will be based on the **quality of the algorithmic reasoning**, not merely on whether you name a known algorithm.

Evidence Expected

A coherent redesigned computational strategy directly addressing the diagnosed bottleneck, supported by appropriate design representation and technical justification.

D. Predict Before Implementing **1 Mark**

Before testing the redesigned solution:

- Predict how its behaviour should differ from the original.
- Identify the performance measure in which you expect improvement.

Evidence Expected

A clear, testable prediction made before experimental validation.

E. Implement and Validate the Redesign **2 Marks**

Implement a prototype of the redesigned strategy wherever feasible.

Using the **same or equivalent test conditions**, compare:

Original Solution → Redesigned Solution

Use appropriate measures such as:

- Comparisons
- Key operations
- Candidate solutions examined
- Distance calculations
- Character comparisons
- Execution time, where relevant

Evidence Expected

Comparable experimental evidence showing the behaviour of the original and redesigned solutions.

F. Defend the Final Design **2 Marks**

Evaluate whether the redesign achieved its intended objective.

Discuss:

- Whether the prediction was supported
- Magnitude of improvement, if any
- New costs or trade-offs
- Situations in which the original solution may still be preferable
- Whether the redesigned solution sufficiently addresses the identified limitation

A redesign that does **not** outperform the original will not automatically be considered unsuccessful if you correctly analyse and explain why the design hypothesis did not produce the expected result.

Evidence Expected

A critical, evidence-based evaluation of the redesigned solution and a defensible final engineering conclusion.

MARK DISTRIBUTION

Question	Assessed Area	Marks

Q1	Selection, Bubble & Insertion Sort	**6**
Q2	Sequential Search	**4**
Q3	Brute-Force String Matching	**6**
Q4	Brute-Force Closest Pair	**6**
Q5	Brute-Force Convex Hull	**7**
Q6	Exhaustive Search	**8**
Q7	**Day-4 Design Challenge – Diagnose, Redesign & Validate**	**13**
	TOTAL	**50**

Submission Requirements

Submit:

- Your assignment report containing the required evidence for Q1–Q7
- Source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Operation counts, comparison counts or other algorithm-specific measures should be treated as **primary experimental evidence**. Execution time may be reported as additional evidence but should not be used as the sole basis for your conclusions.

Assessment Verification

Students may be asked to explain any part of their:

- Algorithmic design
- Source code
- Test-data design
- Experimental results
- Predictions
- Analysis
- Design Challenge

during a brief verification discussion/viva.

The student should therefore be able to **explain and defend the evidence submitted as their own work**.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook algorithms.

It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by **your own mathematical reasoning and experimental evidence**.

Q1 – Sorting Under Different Data Conditions (6 marks)

1.1 Understanding the problem

The fulfilment centre must sort orders by dispatch priority. Selection, Bubble and Insertion Sort all take $\Theta(n^2)$ time on average, so the real question is whether the **arrangement of the incoming data** changes the amount of work. I therefore predict the behaviour first, design inputs that expose the differences, and only then run the experiments.

1.2 Prediction before implementation (n = 1000)

The reasoning behind the predictions is as follows.

- **Selection Sort** always scans the whole unsorted part to find the minimum, and it has no early-exit condition. So its comparisons are fixed at $n(n-1)/2 = 499{,}500$ for every input. It makes at most one swap per pass, so swaps $\leq n-1$.
- **Bubble Sort** (with an early-exit flag) stops after a pass with no swap. The number of passes equals the largest distance any element must travel to the left, because an element moves only one place left per pass. Its swaps equal the number of inversions. A single small value near the end of the list therefore forces many passes, even if the rest is sorted.
- **Insertion Sort** shifts each element past larger elements on its left. Shifts equal the number of inversions, and comparisons ≈ inversions + $(n-1)$. Its work is proportional to how disordered the data is, so it is adaptive.

Table 1: Predicted comparisons for n = 1000 (made before running the code)

Dataset	Selection	Bubble	Insertion
A Random	499,500	≈ 500,000 ($n^2/2$)	≈ 250,000 ($n^2/4$)
B 95% sorted + new batch at the end	499,500	≈ 500,000 (small values must travel far left)	≈ 25,000 (50 items × ~475 shifts)
C Reverse sorted	499,500	499,500	499,500
D Already sorted	499,500	999 (one pass)	999
E Many duplicates (10 levels)	499,500	≈ 500,000	≈ 225,000 (ties are not inversions)

1.3 Design of the test datasets

- **Dataset A – Random (contrast 1):** 1000 priority scores drawn at random from 1 to 100000 (seed 42). Represents the highly disordered case.
- **Dataset B – 95% sorted + new batch (contrast 2):** 950 distinct scores already in order, followed by 50 new random orders appended at the end. This directly models the fulfilment centre's description.
- **Dataset C – Reverse sorted (my edge case):** the random list in descending order, the worst case for any exchange-based algorithm.
- **Extra datasets:** D, the already-sorted list (best case), and E, a list with only 10 distinct values (many ties, which tests stability and equal keys).

1.4 Implementation

All three algorithms count comparisons and swaps. For Insertion Sort the second counter records element shifts (each shift is one data movement). Every output is verified against Python's built-in sorted() by an assertion.

```python
# Q1 - Sorting under different data conditions (function-based, Python 3)
import random, time, csv

def selection_sort(a):
    a = a[:]; comps = swaps = 0
    for i in range(len(a) - 1):
        m = i
        for j in range(i + 1, len(a)):
            comps += 1
            if a[j] < a[m]:
                m = j
        if m != i:
            a[i], a[m] = a[m], a[i]; swaps += 1
    return a, comps, swaps

def bubble_sort(a):                       # with early-exit flag
    a = a[:]; comps = swaps = 0
    for i in range(len(a) - 1):
        swapped = False
        for j in range(len(a) - 1 - i):
            comps += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]; swaps += 1; swapped = True
        if not swapped:
            break
    return a, comps, swaps

def insertion_sort(a):                    # "swaps" column = element shifts
    a = a[:]; comps = shifts = 0
    for i in range(1, len(a)):
        v, j = a[i], i - 1
        while j >= 0:
            comps += 1
            if a[j] > v:
                a[j + 1] = a[j]; shifts += 1; j -= 1
            else:
                break
        a[j + 1] = v
    return a, comps, shifts
```

Screenshot 1: Source code, q1_sorting.py (part 1 of 2)

```python
def make_datasets(n, seed=42):
    r = random.Random(seed)
    rand = [r.randint(1, 100000) for _ in range(n)]
    base = sorted(r.sample(range(1, 100000), int(n * 0.95)))
    batch = [r.randint(1, 100000) for _ in range(n - len(base))]
    return {
        "A Random":              rand,
        "B 95% sorted+new batch": base + batch,
        "C Reverse sorted":      sorted(rand, reverse=True),
        "D Already sorted":      sorted(rand),
        "E Many duplicates":     [r.randint(1, 10) for _ in range(n)],
    }

ALGOS = [("Selection", selection_sort), ("Bubble", bubble_sort),
         ("Insertion", insertion_sort)]

def run(n, rows):
    for dname, data in make_datasets(n).items():
        for aname, f in ALGOS:
            t0 = time.perf_counter()
            out, c, s = f(data)
            ms = (time.perf_counter() - t0) * 1000
            assert out == sorted(data)
            rows.append([n, dname, aname, c, s, round(ms, 2)])

if __name__ == "__main__":
    rows = []
    run(1000, rows)
    print(f"{'Dataset':26}{'Algorithm':11}{'Comparisons':>12}{'Swaps/Shifts':>14}"
          f"{'Time(ms)':>10}")
    for n, d, a, c, s, ms in rows:
        print(f"{d:26}{a:11}{c:>12,}{s:>14,}{ms:>10}")
    print("\nScaling on dataset B (95% sorted + appended batch)")
    srows = []
    for n in (500, 1000, 2000, 4000):
        data = make_datasets(n)["B 95% sorted+new batch"]
        res = [f(data) for _, f in ALGOS]
        srows.append([n] + [x for r_ in res for x in (r_[1], r_[2])])
        print(f"n={n:5}  Sel comps={res[0][1]:>10,}  Bub comps={res[1][1]:>10,}"
              f"  Ins comps={res[2][1]:>9,}  Ins shifts={res[2][2]:>9,}")
    with open("data/q1_raw.csv", "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["n","dataset","algorithm","comparisons","swaps_or_shifts","time_ms"])
        w.writerows(rows)
    with open("data/q1_scaling_B.csv", "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["n","sel_c","sel_s","bub_c","bub_s","ins_c","ins_s"])
        w.writerows(srows)
```

Screenshot 2: Source code, q1_sorting.py (part 2 of 2)

1.5 Observed results

```
Terminal
$ python q1_sorting.py
Dataset                 Algorithm   Comparisons  Swaps/Shifts  Time(ms)
A Random                Selection       499,500           990     17.29
A Random                Bubble          499,329       241,624     35.62
A Random                Insertion       242,617       241,624      17.2
B 95% sorted+new batch  Selection       499,500           945     17.19
B 95% sorted+new batch  Bubble          495,759        22,174     20.56
B 95% sorted+new batch  Insertion        23,173        22,174      1.62
C Reverse sorted        Selection       499,500           502     18.42
C Reverse sorted        Bubble          499,500       499,494      46.3
C Reverse sorted        Insertion       499,500       499,494     33.71
D Already sorted        Selection       499,500             0     16.82
D Already sorted        Bubble              999             0      0.04
D Already sorted        Insertion           999             0      0.06
E Many duplicates       Selection       499,500           908     19.36
E Many duplicates       Bubble          495,584       219,931     35.41
E Many duplicates       Insertion       220,928       219,931     17.93

Scaling on dataset B (95% sorted + appended batch)
n=  500  Sel comps=   124,750  Bub comps=   124,705  Ins comps=    7,379  Ins shifts=    6,880
n= 1000  Sel comps=   499,500  Bub comps=   495,759  Ins comps=   23,173  Ins shifts=   22,174
n= 2000  Sel comps= 1,999,000  Bub comps= 1,997,674  Ins comps=  102,482  Ins shifts=  100,483
n= 4000  Sel comps= 7,998,000  Bub comps= 7,997,895  Ins comps=  385,640  Ins shifts=  381,641
$ _
```

Screenshot 3: Program output of q1_sorting.py

Table 2: Comparisons and swaps/shifts, n = 1000 (observed)

Dataset	Sel comps	Sel swaps	Bub comps	Bub swaps	Ins comps	Ins shifts
A Random	499,500	990	499,329	241,624	242,617	241,624
B 95% sorted+new batch	499,500	945	495,759	22,174	23,173	22,174
C Reverse sorted	499,500	502	499,500	499,494	499,500	499,494
D Already sorted	499,500	0	999	0	999	0
E Many duplicates	499,500	908	495,584	219,931	220,928	219,931

Table 3: Execution time in milliseconds (supporting evidence only)

Dataset	Selection	Bubble	Insertion
A Random	17.29	35.62	17.2
B 95% sorted+new batch	17.19	20.56	1.62
C Reverse sorted	18.42	46.3	33.71
D Already sorted	16.82	0.04	0.06

Dataset	Selection	Bubble	Insertion
E Many duplicates	19.36	35.41	17.93

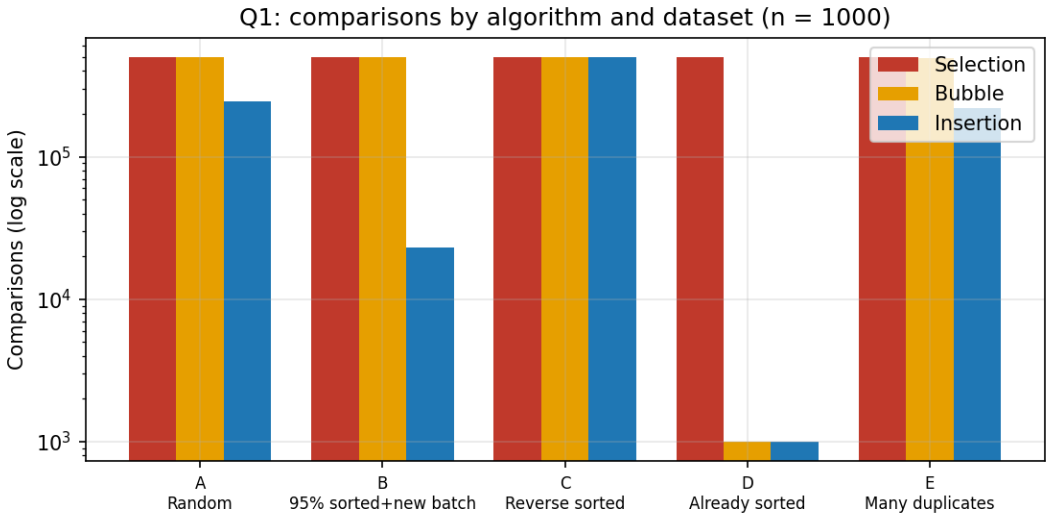


Figure 1: Comparisons by algorithm and dataset (log scale)

Table 4: Scaling on Dataset B (5% of the list is a new batch appended at the end)

n	Selection comps	Bubble comps	Insertion comps	Insertion shifts
500	124,750	124,705	7,379	6,880
1,000	499,500	495,759	23,173	22,174
2,000	1,999,000	1,997,674	102,482	100,483
4,000	7,998,000	7,997,895	385,640	381,641

1.6 Prediction versus observation

- **Selection Sort – prediction confirmed.** Comparisons were 499,500 for all five datasets, even for already-sorted data. Swaps stayed below n (0 to 990).
- **Insertion Sort on B – confirmed.** Predicted about 25,000 comparisons; observed 23,173 with 22,174 shifts. That is about 21.6 times fewer comparisons than Selection Sort and about 10 times faster in time.
- **Bubble Sort on B – prediction confirmed and the most instructive result.** Even though 95% of the data was already in order, Bubble Sort needed 495,759 comparisons, nearly as many as on random data. The 50 new items start at the right end and move only one place left per pass, so about 950 passes are needed before the early-exit flag can fire.

- **Random and reverse data:** Bubble swaps (241,624) equal Insertion shifts (241,624) exactly because both equal the number of inversions in the data. On reverse data both reach $n(n-1)/2$ (observed 499,494, six pairs fewer because six values in the random list are equal).
- **Sorted data (D):** Bubble and Insertion need only 999 comparisons and no swaps, as predicted, while Selection still does the full 499,500.
- **Small deviation:** Selection Sort on reverse data made only 502 swaps, not close to $n-1$. This is allowed by the upper bound I predicted, but the exact figure is lower than I expected: the swap in each pass exchanges two items and so puts both ends right, which roughly halves the swaps.

1.7 Design decision

Recommendation: Insertion Sort

No single algorithm is best for all inputs: Selection Sort is insensitive to the data, Bubble Sort is sensitive but hurt by elements that must move far left, and Insertion Sort adapts to the amount of disorder.

For the fulfilment centre, with about 95% of orders already in order and small batches added periodically, Insertion Sort did about **23 thousand comparisons instead of about 500 thousand** (a 21.6× reduction) on the experimental data.

Additional reasons: it is stable (orders with equal priority keep their arrival order), it sorts in place with O(1) extra memory, and it works well as new batches arrive.

Selection Sort would be preferable only if data writes were extremely expensive (for example flash memory), because it performs at most $n-1$ swaps. That is not the case here.

Limit: the advantage depends on the disorder staying small. If each batch grows with n (as in my scaling table, where the batch is 5% of n), the cost still grows quadratically, so a very large batch would be better sorted separately and then merged.

Q2 – When Is Sequential Search an Appropriate Engineering Choice? (4 marks)

2.1 Experiment design

Each record is a tuple (Tool ID, Technician ID, Issue Time, Work Bay). The search returns the record and the number of comparisons. Three kinds of experiment were designed:

- **Successful searches at different positions:** first record, middle record and last record.
- **Unsuccessful search:** a Tool ID that is not on the list.
- **Different list sizes:** n = 100, 1,000, 10,000 and 100,000, plus the average over 1,000 random successful searches at each size.

Prediction: a successful search at position k costs exactly k comparisons, so the average over random targets is about (n+1)/2; an unsuccessful search costs exactly n. Both grow linearly with n.

2.2 Implementation

```python
# Q2 - Sequential search on tool-issue records
import random, time, csv

def sequential_search(records, tool_id):
    comps = 0
    for rec in records:
        comps += 1
        if rec[0] == tool_id:
            return rec, comps
    return None, comps

def make_records(n):
    r = random.Random(7)
    return [(f"T-{i}", f"TECH-{r.randint(1,300)}",
            f"{r.randint(0,23):02d}:{r.randint(0,59):02d}",
             f"BAY-{r.randint(1,12)}") for i in range(1, n + 1)]

if __name__ == "__main__":
    rows = []
    print(f"{'n':>8}{'first':>8}{'middle':>9}{'last':>9}{'absent':>9}"
          f"{'avg(1000 random hits)':>24}{'time/search(us,avg)':>21}")
    for n in (100, 1000, 10000, 100000):
        recs = make_records(n)
        first = sequential_search(recs, "T-1")[1]
        mid = sequential_search(recs, f"T-{n//2}")[1]
        last = sequential_search(recs, f"T-{n}")[1]
        absent = sequential_search(recs, "T-999999999")[1]
        r = random.Random(1)
        keys = [f"T-{r.randint(1, n)}" for _ in range(1000)]
        t0 = time.perf_counter()
        avg = sum(sequential_search(recs, k)[1] for k in keys) / 1000
        us = (time.perf_counter() - t0) / 1000 * 1e6
        rows.append([n, first, mid, last, absent, round(avg, 1), round(us, 1)])
        print(f"{n:>8,}{first:>8}{mid:>9,}{last:>9,}{absent:>9,}{avg:>24,.1f}{us:>21.1f}")
    with open("data/q2_raw.csv", "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["n","first","middle","last","absent","avg_success","us_per_search"])
        w.writerows(rows)
```

Screenshot 4: Source code, q2_sequential.py (part 1 of 2)

```
    # Operating-condition model: cost per comparison measured from the n=100,000 run
    t_cmp = rows[-1][6] / rows[-1][5] * 1e-6             # seconds per comparison
    print(f"\nMeasured cost per comparison ~ {t_cmp*1e9:.0f} ns")
    recs = make_records(100000); idx = {r_[0]: r_ for r_ in recs}
    keys = [f"T-{random.randint(1, 100000)}" for _ in range(1000)]
    t0 = time.perf_counter()
    for k in keys: idx.get(k)
    print(f"Dictionary (hash index) lookup, n=100,000: "
          f"{(time.perf_counter()-t0)/1000*1e6:.2f} us/search")

    print(f"\n{'Scenario':36}{'n':>8}{'req/s':>8}{'latency(ms)':>13}{'CPU load':>10}")
    for name, n, rate in [("1 hangar (normal)", 300, 0.05),
                          ("1 hangar (shift-change peak)", 300, 2),
                          ("20 facilities (normal)", 6000, 1),
                          ("20 facilities (peak)", 6000, 40),
                          ("20 facilities, records x10, peak", 60000, 40),
                          ("20 facilities + barcode scanners", 60000, 2000)]:
        lat = n * t_cmp * 1000                           # worst case = n comparisons
        load = rate * (n / 2) * t_cmp * 100              # average successful search
        print(f"{name:36}{n:>8,}{rate:>8}{lat:>13.3f}{load:>9.2f}%")
```

Screenshot 5: Source code, q2_sequential.py (part 2 of 2)

2.3 Results

```
Terminal
$ python q2_sequential.py
       n   first   middle     last   absent   avg(1000 random hits)  time/search(us,avg)
     100       1       50      100      100                   51.1                  1.1
   1,000       1      500    1,000    1,000                  510.1                 13.3
  10,000       1    5,000   10,000   10,000                5,053.3                194.9
 100,000       1   50,000  100,000  100,000               50,438.5               1411.4

Measured cost per comparison ~ 28 ns
Dictionary (hash index) lookup, n=100,000: 0.27 us/search

Scenario                                n   req/s  latency(ms)  CPU load
1 hangar (normal)                     300    0.05        0.008     0.00%
1 hangar (shift-change peak)          300       2        0.008     0.00%
20 facilities (normal)              6,000       1        0.168     0.01%
20 facilities (peak)                6,000      40        0.168     0.34%
20 facilities, records x10, peak   60,000      40        1.679     3.36%
20 facilities + barcode scanners   60,000    2000        1.679   167.90%
$ _
```

Screenshot 6: Program output of q2_sequential.py

Table 5: Comparison counts for sequential search

List size n	First	Middle	Last	Unsuccessful	Average successful	Time per search (µs)
100	1	50	100	100	51.1	1.1
1,000	1	500	1,000	1,000	510.1	13.3
10,000	1	5,000	10,000	10,000	5,053.3	194.9
100,000	1	50,000	100,000	100,000	50,438.5	1411.4

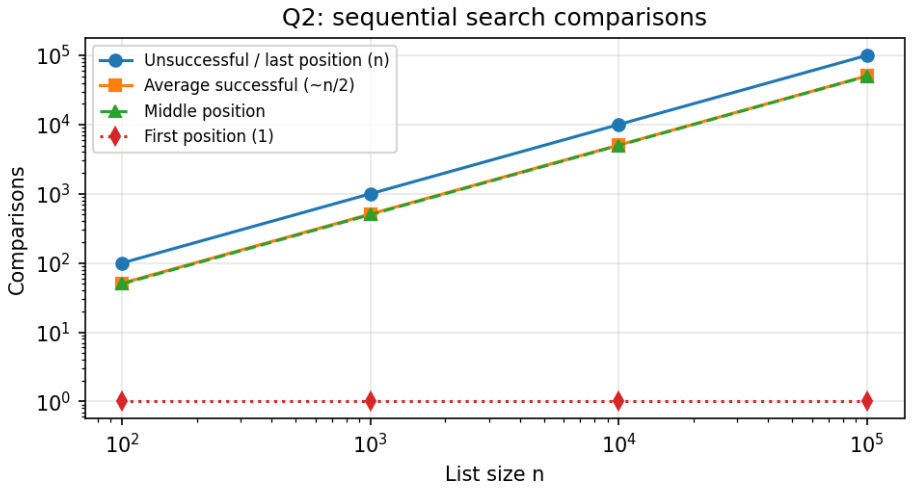


Figure 2: Sequential-search comparisons against list size (log–log)

The evidence matches the prediction exactly: the first, middle and last positions cost 1, n/2 and n comparisons, an unsuccessful search always costs n, and the average successful search is within 1% of n/2 (for example 50,438 at n = 100,000). The time per search grows in proportion to n (about 28 ns per comparison in this environment).

2.4 When is sequential search reasonable?

Sequential search needs no extra structure and no ordering, so it is the simplest choice for a short list that changes constantly (tools are issued and returned all the time). To place a boundary, I defined two acceptance criteria and used the measured 28 ns per comparison:

- **Latency:** the worst-case lookup (n comparisons) should finish within 10 ms. That allows n up to about 350,000 records.
- **Load:** the average search costs n/2 comparisons, so sustained load = rate × (n/2) × 28 ns. Keeping this below 25% of one processor core requires **rate × n ≤ about 1.8 × 10^7**.

The request rates below are assumptions for the model (the assignment does not give them); the comparison cost is measured.

Table 6: Operating-condition analysis (assumed request rates)

Scenario	Active records n	Requests/s	Worst latency (ms)	CPU load	Verdict
1 hangar, normal	300	0.05	0.008	0.00%	Sequential is fine
1 hangar, shift-change peak	300	2	0.008	0.00%	Sequential is fine
20 facilities, normal	6,000	1	0.168	0.01%	Sequential is fine

Scenario	Active records n	Requests/s	Worst latency (ms)	CPU load	Verdict
20 facilities, peak	6,000	40	0.168	0.34%	Sequential is fine
20 facilities, records × 10, peak	60,000	40	1.679	3.36%	Acceptable, little headroom
20 facilities + automatic barcode scanners	60,000	2,000	1.679	167.9%	Fails: needs an index

2.5 Changed condition: 20 maintenance facilities

Original recommendation: keep Sequential Search for the single hangar. With about 300 active records the lookup takes microseconds.

Re-evaluation: expanding to 20 facilities multiplies both the records and the requests, so the product rate × n grows by about 400 times, but the evidence shows it is still roughly 70 times below my boundary (240,000 against 1.8×10^7 at peak). Sequential search therefore **remains acceptable** if each facility keeps its own list or if the central list stays in the thousands of records with human-triggered lookups.

When I would stop recommending Sequential Search

(1) The active list grows beyond about 10^5 records, where a worst-case lookup passes 1 ms and keeps rising linearly; or

(2) lookups become automated and frequent (barcode scanners, dashboards, audit scripts), so that rate × n exceeds about 1.8×10^7; or

(3) unsuccessful searches become common, because each one always costs the full n comparisons.

Replacement: a hash index (Python dictionary) keyed by Tool ID. It supports insertion on issue and deletion on return in O(1) on average, which suits a list that changes continuously. In my test it took 0.27 µs per lookup against 1,411 µs for sequential search at n = 100,000, about 5,000 times faster. A sorted list with binary search is a poor fit here because every issue or return would cost O(n) to keep it sorted.

Because the extra memory is only one index entry per record, adopting the dictionary when the system is centralised is cheap insurance, even though the measured evidence shows sequential search still copes at 20 facilities.

Q3 – Engineer the Behaviour of Brute-Force String Matching (6 marks)

3.1 Algorithm

Brute-force matching aligns the pattern with every position of the text (n − m + 1 alignments) and compares characters from left to right until a mismatch or a complete match. After a mismatch it shifts by exactly one position and starts again, **discarding everything it learned**. The function returns all match positions and the total number of character comparisons.

```python
# Q3 - Brute-force string matching: engineered inputs
import random, csv

def brute_force_match(text, pat):
    """Scans every alignment; returns (list of match positions, character comparisons)."""
    n, m = len(text), len(pat)
    comps, matches = 0, []
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comps += 1
            if text[i + j] != pat[j]:
                break
            j += 1
        if j == m:
            matches.append(i)
    return matches, comps

if __name__ == "__main__":
    n = 10000
    r = random.Random(11)
    tests = []
    # T1 best case: first pattern character never occurs in text -> 1 comparison per alignment
    tests.append(("T1 Best case", "A" * n, "B" + "A" * 9, lambda n, m: n - m + 1))
    # T2 engineered worst case: m-1 matching characters then a mismatch at every alignment
    tests.append(("T2 Worst case", "A" * n, "A" * 9 + "B", lambda n,
            m: m * (n - m + 1)))
    # T3 own condition: random 4-letter (DNA-like) text, random pattern
    text3 = "".join(r.choice("ACGT") for _ in range(n))
    tests.append(("T3 Random 4-letter", text3, "ACGTTGCA",
                  lambda n, m: (n - m + 1) * sum(0.25 ** k for k in range(m))))
    rows = []
    print(f"{'Test':20}{'n':>7}{'m':>4}{'Alignments':>12}{'Predicted':>12}"
          f"{'Observed':>10}{'Cmp/alignment':>15}{'Matches':>9}")
    for name, text, pat, pred in tests:
        matches, c = brute_force_match(text, pat)
        n_, m_ = len(text), len(pat)
        rows.append([name, n_, m_, n_ - m_ + 1, round(pred(n_, m_)), c,
                     len(matches)])
        print(f"{name:20}{n_:>7,}{m_:>4}{n_-m_+1:>12,}{pred(n_, m_):>12,.0f}"
              f"{c:>10,}{c/(n_-m_+1):>15.2f}{len(matches):>9}")
```

Screenshot 7: Source code, q3_string.py (part 1 of 2)

```python
    print("\nSame text length (n = 10,000), pattern length varied - worst-case family")
    print(f"{'m':>4}{'Predicted m(n-m+1)':>22}{'Observed':>12}")
    sweep = []
    for m in (2, 5, 10, 20, 50):
        _, c = brute_force_match("A" * n, "A" * (m - 1) + "B")
        sweep.append([m, m * (n - m + 1), c])
        print(f"{m:>4}{m*(n-m+1):>22,}{c:>12,}")
    with open("data/q3_raw.csv", "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["test","n","m","alignments","predicted","observed","matches"])
        w.writerows(rows)
    with open("data/q3_sweep.csv", "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["m","predicted","observed"])
        w.writerows(sweep)
```

Screenshot 8: Source code, q3_string.py (part 2 of 2)

3.2 Test design and predictions (made before running)

Table 7: Designed tests (text length n = 10,000)

Test	Text	Pattern	Prediction and reason
T1 Little work	"A" repeated 10,000 times	"BAAAAAAAAA" (m = 10)	The first pattern character never matches, so every alignment stops after 1 comparison: n − m + 1 = 9,991.
T2 Engineered heavy work	"A" repeated 10,000 times	"AAAAAAAAAB" (m = 10)	Every alignment matches 9 characters and fails on the 10th, costing m comparisons: m(n − m + 1) = 99,910.
T3 My own condition	Random 4-letter (A, C, G, T) text	"ACGTTGCA" (m = 8)	Each comparison matches with probability 1/4, so the expected comparisons per alignment are 1 + 1/4 + 1/16 + … ≈ 1.333, giving about 9,993 × 1.333 ≈ 13,324.

3.3 Observed results

```
Terminal
$ python q3_string.py
Test                     n   m  Alignments   Predicted  Observed  Cmp/alignment  Matches
T1 Best case        10,000  10       9,991       9,991     9,991           1.00        0
T2 Worst case       10,000  10       9,991      99,910    99,910          10.00        0
T3 Random 4-letter  10,000   8       9,993      13,324    13,348           1.34        0

Same text length (n = 10,000), pattern length varied - worst-case family
   m    Predicted m(n-m+1)    Observed
   2               19,998      19,998
   5               49,980      49,980
  10               99,910      99,910
  20              199,620     199,620
  50              497,550     497,550
$ _
```

Screenshot 9: Program output of q3_string.py

Table 8: Predicted and observed character comparisons

Test	Alignments	Predicted	Observed	Comparisons per alignment
T1 Best case	9,991	9,991	9,991	1.00
T2 Worst case	9,991	99,910	99,910	10.00
T3 Random 4-letter	9,993	13,324	13,348	1.34

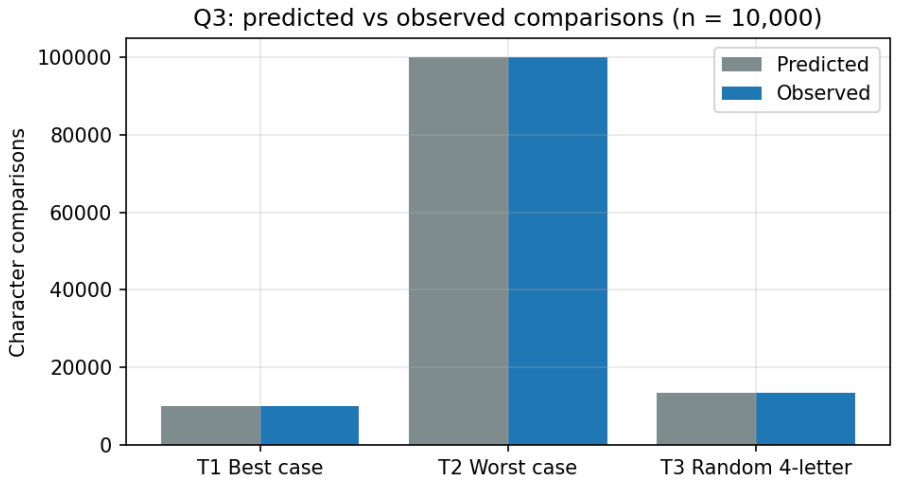


Figure 3: Predicted versus observed character comparisons

T1 and T2 matched the prediction exactly (9,991 and 99,910). T3 gave 13,348 against a predicted 13,324, an error of only 0.18%; the small difference arises because one random text is a sample, not the average.

Table 9: Same text length, different pattern length (worst-case family)

Pattern length m	Predicted m(n − m + 1)	Observed
2	19,998	19,998
5	49,980	49,980
10	99,910	99,910
20	199,620	199,620
50	497,550	497,550

3.4 Inquiry

Why does one input need much more work than another?

T1 and T2 have identical text and almost identical patterns, and both have 9,991 alignments, yet T2 needs exactly ten times more character comparisons (99,910 against 9,991). The work per alignment is decided by **how far the match gets before failing**. In T1 the mismatch comes immediately; in T2 it comes at the last character, so almost the whole pattern is examined at every alignment.

Which characteristics create repeated comparisons?

- A text made of long runs of the same character or a repeating prefix (for example, "AAAA…").
- A pattern that begins with a long stretch that also occurs in the text and differs only at the end ("AAAAAAAAAB").
- A small alphabet. In T3 (4 letters) the average cost was 1.34 comparisons per alignment; with a 26-letter alphabet it would fall towards 1.04, because accidental partial matches are rarer.
- Longer patterns raise the worst case in proportion to m, as the sweep table shows (19,998 comparisons for m = 2 up to 497,550 for m = 50 on the same text).

The underlying cause is that brute force forgets: after a mismatch at alignment i it shifts by one and re-reads text characters it has already compared. In T2 each text character is compared up to 10 times.

What this reveals about judging an algorithm only by input size

All tests used n = 10,000, yet the effort ranged from 9,991 to 99,910 comparisons, and the sweep shows 19,998 to 497,550 for the same n. The input size alone therefore cannot predict the running time. The structure and content of the input (here, the match prefix and the alphabet) matter, which is why algorithms are described by best, average and worst cases: brute-force matching is $\Omega(n)$ in the best case, about $\Theta(n)$ on random text, and $O(nm)$ in the worst case.

Q4 – Closest Pair and the Cost of Examining Every Possibility (6 marks)

4.1 Algorithm design

- **Which pairs must be examined?** Every unordered pair of distinct drones, because any pair could be the closest and nothing is known in advance about the positions.
- **How are duplicates avoided?** The distance from D_i to D_j equals the distance from D_j to D_i, and a drone is not compared with itself. By letting the inner index start at i + 1 (j > i), each unordered pair is visited exactly once.
- **Distance:** squared distance $dx^2 + dy^2$ is compared instead of the square root, which gives the same ordering with less work; sqrt is taken only once at the end.

Derivation of the number of unique pairs

Drone 1 is paired with the n − 1 later drones, drone 2 with n − 2, and so on, down to drone n − 1 with 1. Summing: $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$. The same result comes from $C(n, 2) = n!/(2!(n-2)!) = n(n-1)/2$. Counting ordered pairs would give $n(n-1)$, exactly twice as many, which is the duplicate work avoided. Since $n(n-1)/2 \approx n^2/2$, **doubling n multiplies the work by about 4**, so the algorithm is $\Theta(n^2)$.

```
ALGORITHM BruteForceClosestPair(D[0..n-1])
  best ← ∞
  for i ← 0 to n-2:
    for j ← i+1 to n-1:
      d2 ← (xi - xj)^2 + (yi - yj)^2
      if d2 < best: best ← d2; pair ← (i, j)
  return pair, sqrt(best)
```

```python
# Q4 - Brute-force closest pair of drones
import random, time, csv

def closest_pair_bf(pts):
    n = len(pts)
    best, pair, pairs, dists = float("inf"), None, 0, 0
    for i in range(n - 1):                  # only j > i : each unordered pair once
        for j in range(i + 1, n):
            pairs += 1
            dx = pts[i][0] - pts[j][0]
            dy = pts[i][1] - pts[j][1]
            d2 = dx * dx + dy * dy          # squared distance (sqrt not needed to compare)
            dists += 1
            if d2 < best:
                best, pair = d2, (i, j)
    return best ** 0.5, pair, pairs, dists

def make_points(n, seed=5):
    r = random.Random(seed)
    return [(r.uniform(0, 1000), r.uniform(0, 1000)) for _ in range(n)]

if __name__ == "__main__":
    rows = []
    print(f"{'n':>6}{'n(n-1)/2':>14}{'pairs examined':>16}{'dist calcs':>12}"
          f"{'time(s)':>9}{'time/pair(ns)':>15}")
    for n in (100, 500, 1000, 2000, 4000, 6000):
        pts = make_points(n)
        t0 = time.perf_counter()
        d, pair, pairs, dists = closest_pair_bf(pts)
        t = time.perf_counter() - t0
        rows.append([n, n * (n - 1) // 2, pairs, dists, round(t, 4)])
        print(f"{n:>6,}{n*(n-1)//2:>14,}{pairs:>16,}{dists:>12,}{t:>9.3f}"
              f"{t/pairs*1e9:>15.1f}")
    # growth ratio when n doubles, and extrapolation  t = c * n^2
    c = sum(r_[4] / r_[0] ** 2 for r_ in rows[2:]) / len(rows[2:])
    print(f"\nFitted model: t = {c:.3e} * n^2")
    for limit in (0.1, 1, 10, 60):
        print(f"Largest n finishing within {limit:>4} s : about "
              f"{int((limit / c) ** 0.5):,} drones")
    with open("data/q4_raw.csv", "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["n","n(n-1)/2","pairs_examined","distance_calcs","time_s"])
        w.writerows(rows)
```

Screenshot 10: Source code, q4_closest_pair.py

4.2 Experiments

Drones were placed uniformly at random in a 1000 × 1000 area (seed 5), with n = 100, 500, 1,000, 2,000, 4,000 and 6,000. **Prediction:** pairs examined = n(n−1)/2 exactly, distance calculations = pairs examined, and time ∝ n^2.

```
Terminal
$ python q4_closest_pair.py
     n      n(n-1)/2  pairs examined  dist calcs  time(s)  time/pair(ns)
   100         4,950           4,950       4,950    0.001          116.8
   500       124,750         124,750     124,750    0.016          130.2
 1,000       499,500         499,500     499,500    0.060          119.3
 2,000     1,999,000       1,999,000   1,999,000    0.211          105.5
 4,000     7,998,000       7,998,000   7,998,000    0.855          107.0
 6,000    17,997,000      17,997,000  17,997,000    1.882          104.5

Fitted model: t = 5.452e-08 * n^2
Largest n finishing within  0.1 s : about 1,354 drones
Largest n finishing within    1 s : about 4,282 drones
Largest n finishing within   10 s : about 13,543 drones
Largest n finishing within   60 s : about 33,175 drones
$ _
```

Screenshot 11: Program output of q4_closest_pair.py

Table 10: Pairs examined, distance calculations and time

n	n(n−1)/2 (theory)	Pairs examined	Distance calcs	Time (s)
100	4,950	4,950	4,950	0.0006
500	124,750	124,750	124,750	0.0162
1,000	499,500	499,500	499,500	0.0596
2,000	1,999,000	1,999,000	1,999,000	0.2109
4,000	7,998,000	7,998,000	7,998,000	0.8555
6,000	17,997,000	17,997,000	17,997,000	1.8816

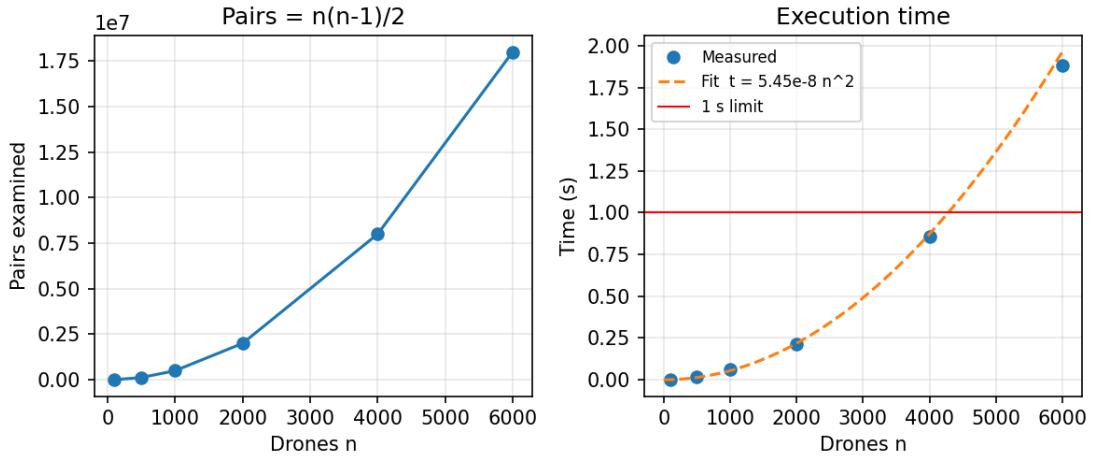


Figure 4: Pairs examined and execution time against n

The theoretical and measured pair counts are identical for every n. From n = 2,000 to n = 4,000 the pair count grows by a factor of 4.0 and the time by about 4.1, and the time per pair stays roughly constant (about 105 ns), so the growth follows n^2.

4.3 Scalability inquiry

Acceptability criterion: the monitoring system refreshes drone positions about once per second, so I define an acceptable response time as **at most 1 second** for one complete closest-pair check. A tighter 0.1 s target is also given for comparison.

Fitting the measurements gives $t \approx 5.45 \times 10^{-8} \times n^2$ seconds.

Table 11: Largest drone count within each time limit (extrapolated from the fit)

Time limit	Largest n (approx.)
0.1 s	1,350
1 s (my criterion)	4,300

Time limit	Largest n (approx.)
10 s	13,500
60 s	33,000

The measured run at n = 6,000 already took 1.88 s, above my limit. **The brute-force approach therefore becomes impractical at about 4,000 drones** in this environment. Because a growth of n^2 is involved, a machine ten times faster would raise this limit only by a factor of about 3.2.

Source of the inefficiency and what a redesign must do

The cost comes from the nested loop that evaluates **every** pair, although only one pair is needed. Most pairs involve drones that are far apart and could never be the answer, yet the algorithm pays the same price for them as for a close pair, and it uses nothing it has learned (the best distance so far, or the ordering of the positions) to skip any of them. A redesigned strategy must (a) avoid computing the distance for pairs that can be ruled out cheaply, and (b) use a property such as spatial ordering or a lower bound on the distance, so that the number of candidate pairs stops growing as n^2. This is carried out in Q7.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle (7 marks)

5.1 From geometric principle to computation

The given principle is: a line through two points may be part of the outer boundary if all remaining points lie on one side of it. Each phrase has to become a computable test.

How two points define a line

For $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$, the line is **ax + by = c** with $a = y_2 - y_1$, $b = x_1 - x_2$ and $c = x_1y_2 - y_1x_2$.

How the side of another point is found

For a point P = (x, y), evaluate **v = ax + by − c**. If v > 0, P is on one side; if v < 0, it is on the other side; if v = 0, P lies exactly on the line. This works because ax + by − c is a signed measure of distance from the line, and its sign changes when the line is crossed.

How the other points are examined and what identifies a hull edge

For each pair (P_i, P_j), every other point P_k is evaluated. The pair is a **hull edge** if all values v have the same sign. Points with v = 0 (collinear points) need care: if such a point lies between P_i and P_j the edge is still valid, but if it lies outside the segment the true edge is longer, so this pair is rejected.

How the final boundary is constructed

The accepted edges are stored as an adjacency list (each hull vertex has exactly two neighbours). Starting at the lowest-leftmost vertex, the program follows neighbours until it returns to the start, which gives the boundary vertices in order. Interior points never appear in any accepted edge.

5.2 Pseudocode (written before implementation)

```
ALGORITHM BruteForceConvexHull(P[0..n-1])
  edges ← empty list
  for i ← 0 to n-2:
    for j ← i+1 to n-1:
      a ← yj - yi;  b ← xi - xj;  c ← xi*yj - yi*xj
      positive ← false;  negative ← false;  ok ← true
      for k ← 0 to n-1  (k ≠ i, k ≠ j):
        v ← a*xk + b*yk - c
        if v > 0: positive ← true
        else if v < 0: negative ← true
        else if Pk is outside segment PiPj: ok ← false; stop
```

```
        if positive and negative: ok ← false; stop
    if ok: add (i, j) to edges
build adjacency lists from edges
walk from the lowest-leftmost vertex along the neighbours
return the ordered boundary vertices
```

5.3 Implementation

```python
# Q5 - Brute-force convex hull from the "all other points on one side" principle
import random, time, csv

def side(p1, p2, p):
    """Line through p1,p2:  a*x + b*y = c.  Returns a*x + b*y - c for point p."""
    a = p2[1] - p1[1]
    b = p1[0] - p2[0]
    c = p1[0] * p2[1] - p1[1] * p2[0]
    return a * p[0] + b * p[1] - c

def on_segment(p1, p2,
        p):                 # p is collinear with p1,p2 - is it between them?
    return (min(p1[0], p2[0]) <= p[0] <= max(p1[0], p2[0]) and
            min(p1[1], p2[1]) <= p[1] <= max(p1[1], p2[1]))

def hull_edges(pts):
    n, edges, tests = len(pts), [], 0
    for i in range(n - 1):
        for j in range(i + 1, n):
            pos = neg = False
            ok = True
            for k in range(n):
                if k == i or k == j:
                    continue
                tests += 1
                v = side(pts[i], pts[j], pts[k])
                if v > 0: pos = True
                elif v < 0: neg = True
                elif not on_segment(pts[i], pts[j], pts[k]):
                    ok = False
                    break      # collinear but outside segment: edge is longer
                if pos and neg:
                    ok = False
                    break      # points on both sides: not a boundary edge
            if ok:
                edges.append((i, j))
    return edges, tests
```

Screenshot 12: Source code, q5_hull.py (part 1 of 3)

```python
def build_boundary(pts, edges):
    """Chain accepted edges into an ordered polygon (vertices only)."""
    if not edges:
        return []
    adj = {}
    for i, j in edges:
        adj.setdefault(i, []).append(j); adj.setdefault(j, []).append(i)
    start = min(adj, key=lambda k: pts[k])
    if len(edges) == 1:
        return [pts[start],
                pts[edges[0][0] if edges[0][1] == start else edges[0][1]]]
    order, prev, cur = [start], None, start
    while True:
        nxt = [v for v in adj[cur] if v != prev][0] if prev is not None else adj[cur][0]
        if nxt == start: break
        order.append(nxt); prev, cur = cur, nxt
    return [pts[k] for k in order]

def monotone_chain(points):                # independent checker only (not the answer)
    P = sorted(set(points))
    if len(P) <= 2: return P
    cr = lambda o, a, b: (a[0]-o[0])*(b[1]-o[1]) - (a[1]-o[1])*(b[0]-o[0])
    lo, up = [], []
    for p in P:
        while len(lo) >= 2 and cr(lo[-2], lo[-1], p) <= 0: lo.pop()
        lo.append(p)
    for p in reversed(P):
        while len(up) >= 2 and cr(up[-2], up[-1], p) <= 0: up.pop()
        up.append(p)
    return lo[:-1] + up[:-1]

SET1 = [(10, 10), (90, 15), (95, 70), (50, 95), (5, 60), (30, 30), (45, 50), (60,
        40),
        (70, 65), (25, 55), (55, 25), (40, 70), (75, 35), (20, 20), (65, 80), (35,
                45)]
SET2 = [(0, 0), (10, 0), (10, 10), (0, 10),            # square corners
        (5, 0), (10, 5), (5, 10), (0,
                5),             # collinear points on every edge
        (5, 5), (3, 3), (7, 7), (2, 8)]                # interior points
SET3 = [(i,
        i * i) for i in range(8)]                  # parabola: every point is a vertex
SET4 = [(i, 2 * i) for i in range(6)]                  # all collinear (degenerate)
```

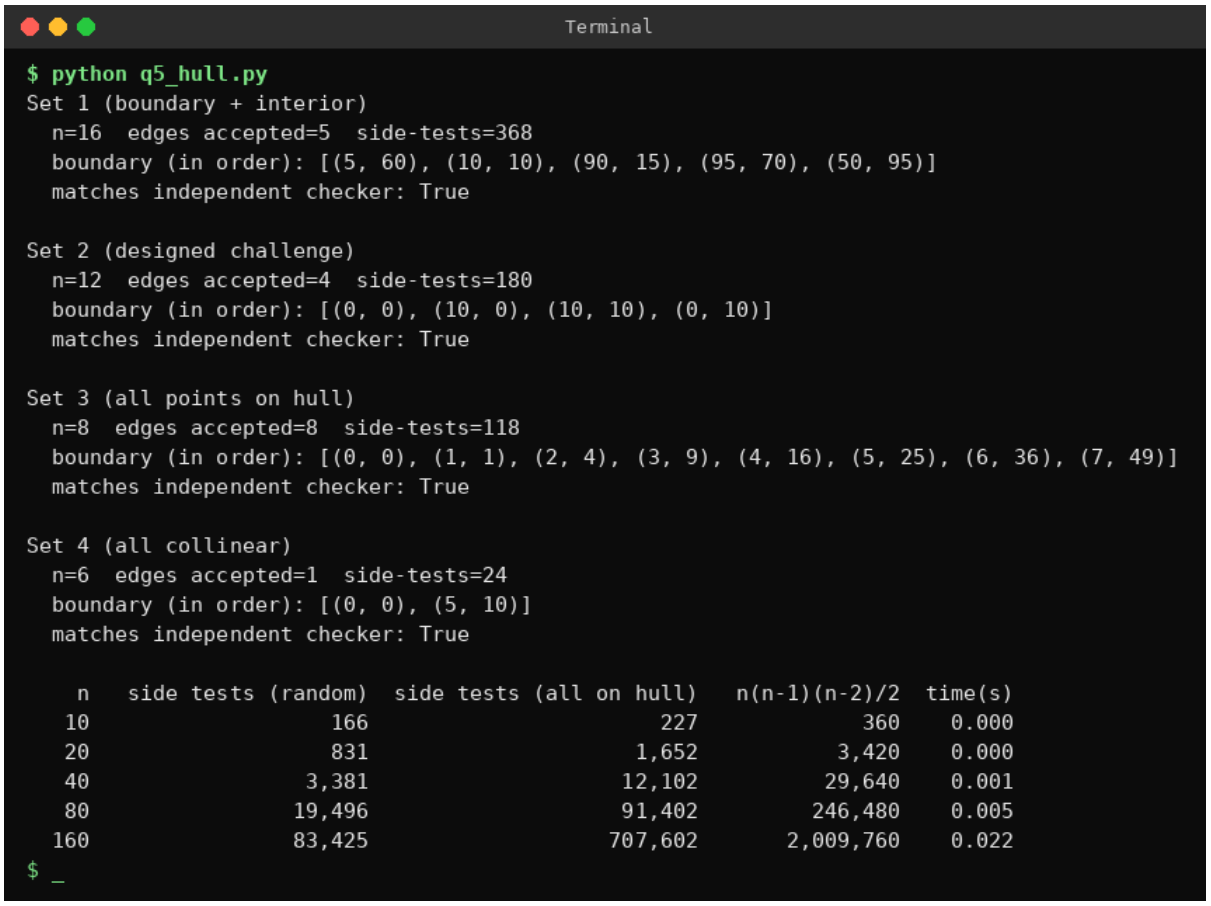
Screenshot 13: Source code, q5_hull.py (part 2 of 3)

```python
if __name__ == "__main__":
    for name, pts in [("Set 1 (boundary + interior)", SET1),
            ("Set 2 (designed challenge)", SET2),
                    ("Set 3 (all points on hull)",
                            SET3), ("Set 4 (all collinear)", SET4)]:
        edges, tests = hull_edges(pts)
        hull = build_boundary(pts, edges)
        ok = set(hull) == set(monotone_chain(pts))
        print(f"{name}\n  n={len(pts)}  edges accepted={len(edges)}  side-tests={tests}")
        print(f"  boundary (in order): {hull}\n  matches independent checker: {ok}\n")
    print(f"{'n':>5}{'side tests (random)':>22}{'side tests (all on hull)':>26}"
          f"{'n(n-1)(n-2)/2':>16}{'time(s)':>9}")
    rows = []
    for n in (10, 20, 40, 80, 160):
        r = random.Random(n)
        rp = [(r.randint(0, 10000), r.randint(0, 10000)) for _ in range(n)]
        t0 = time.perf_counter()
        _, t1 = hull_edges(rp)
        dt = time.perf_counter() - t0
        circle = [(i, i * i) for i in range(n)]
        _, t2 = hull_edges(circle)
        rows.append([n, t1, t2, n * (n - 1) * (n - 2) // 2, round(dt, 4)])
        print(f"{n:>5}{t1:>22,}{t2:>26,}{n*(n-1)*(n-2)//2:>16,}{dt:>9.3f}")
    with open("data/q5_raw.csv", "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["n","tests_random","tests_all_on_hull","upper_bound","time_s"])
        w.writerows(rows)
```

Screenshot 14: Source code, q5_hull.py (part 3 of 3)

To avoid trusting my own program blindly, the results are also compared with an independent checker (the standard monotone-chain method). The checker is used only for validation and is not part of the solution.

5.4 Tests and visualisation



Screenshot 15: Program output of q5_hull.py

Table 12: Test configurations

Test	n	Edges accepted	Side tests	Agrees with checker
Set 1: boundary + interior points	16	5	368	Yes
Set 2: designed challenge (collinear boundary points + interior)	12	4	180	Yes
Set 3: every point is a hull vertex (parabola)	8	8	118	Yes
Set 4: all points collinear (degenerate)	6	1	24	Yes

Set 2 was designed to break a naive implementation. Eight of its points are on the boundary of a square, but four of them (the edge midpoints) are collinear with two corners. A program that only tests “all remaining points on one side” with strict inequalities would reject the true edges (because a collinear point gives v = 0), and one with non-strict inequalities would accept overlapping edges. My segment test handles it: the four corner edges are accepted, the midpoints are correctly not reported as vertices, and the three interior points are

ignored.

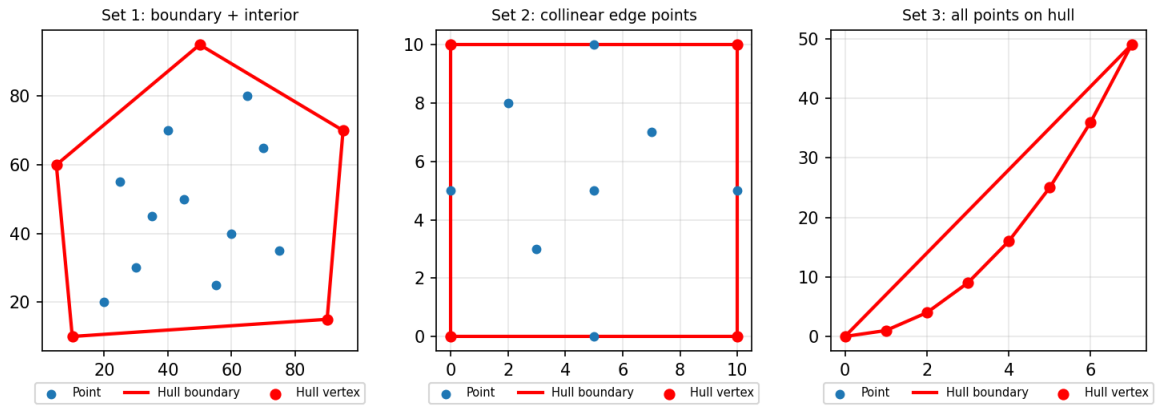


Figure 5: Boundary found by the brute-force algorithm (red) for Sets 1, 2 and 3

5.5 Growth of computational work

There are n(n−1)/2 pairs and each is tested against up to n − 2 other points, so the work is at most n(n−1)(n−2)/2 side tests, i.e. **O(n³)**. Doubling n should multiply the worst case by about 8.

Table 13: Side tests against n

n	Random points	All points on hull	Upper bound n(n−1)(n−2)/2	Time, random (s)
10	166	227	360	0.0001
20	831	1,652	3,420	0.0002
40	3,381	12,102	29,640	0.0009
80	19,496	91,402	246,480	0.0047
160	83,425	707,602	2,009,760	0.0217

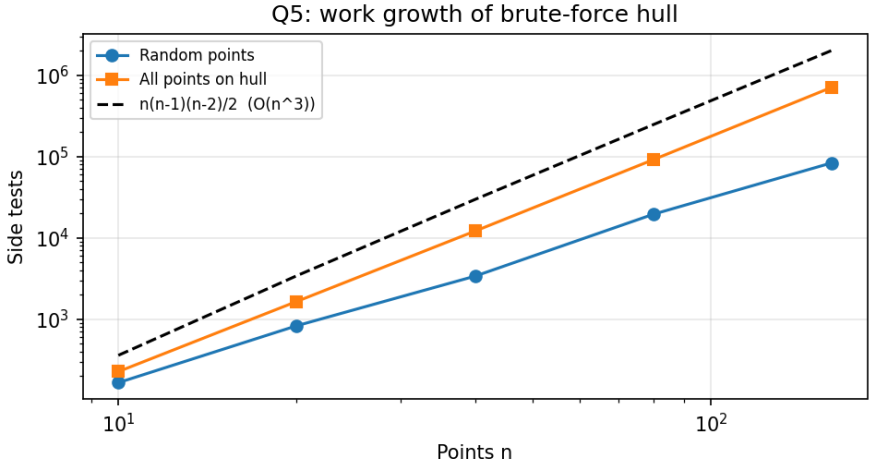


Figure 6: Side tests against n (log–log), with the n³ bound

When every point is on the hull, the growth factor per doubling of n is 7.3, 7.6 and 7.7 (1,652 → 12,102 → 91,402 → 707,602), approaching the predicted 8, so the cubic behaviour is confirmed. For random points the early-exit rule (stop once points are found on both sides) cuts the work to about 4% of the bound at n = 160, because most pairs are rejected after a few tests. The worst case, however, is still cubic.

5.6 Correctness inquiry

Why does the geometric test show that an accepted edge lies on the outer boundary?

A line $ax + by = c$ divides the plane into two closed half-planes. If every point of the set satisfies $ax + by - c \geq 0$ (or every point satisfies ≤ 0), then all the points lie in one closed half-plane.

A half-plane is convex, so any convex combination of the points also lies in it. The convex hull is built entirely from such combinations, so the **whole hull lies in that half-plane**, and the line touches the hull at P_i and P_j. Such a line is called a supporting line.

The segment P_iP_j lies on this line and belongs to the hull (it is a convex combination of two points of the set). Any point of the segment has points of the plane on the other side of the line arbitrarily close to it, but no hull point exists there. So no point of the segment can be interior, and the segment must be part of the boundary.

Completeness (no edge is missed): every edge of the convex polygon has such a supporting line through its two end points, so it passes the test. **Collinear points:** if a point lies on the line outside the segment, the segment is only part of a longer edge, which is why the pair is rejected, and the edge is accepted once between the two extreme points.

Interior points are never edges: for a segment joining an interior point to another point, some points lie on both sides of the line, so the test fails. The assumption is that the points are distinct.

Q6 – Exhaustive Search and Combinatorial Growth (8 marks)

6.1 Design of the exhaustive search

- **Representing a candidate:** a package set is a bit string of length n. Bit i = 1 means package i is loaded. Since a bit string is also an integer, the integers 0 to $2^n - 1$ represent all possible loads (for example 01101 means packages 1, 3 and 4).
- **Generating candidates:** a single loop over mask = 0 … $2^n - 1$ produces every subset exactly once, with nothing missed and nothing repeated.
- **Checking feasibility:** the weights of the selected packages are added, and the candidate is feasible only if the total is ≤ payload limit.
- **Evaluating feasible candidates:** their values are added to give the total humanitarian value.
- **Retaining the best:** the best value and its mask are updated whenever a feasible candidate beats the best so far.

```
ALGORITHM ExhaustiveAirlift(w[0..n-1], v[0..n-1], capacity)
 bestValue ← -1;  bestMask ← 0
 for mask ← 0 to 2^n - 1:
  totalW ← 0;  totalV ← 0
  for i ← 0 to n-1:
   if bit i of mask is 1: totalW ← totalW + w[i];  totalV ← totalV + v[i]
  if totalW ≤ capacity and totalV > bestValue:
   bestValue ← totalV;  bestMask ← mask
 return bestValue, bestMask
```

```python
# Q6 - Exhaustive search for the airlift (0/1 selection) problem
import random, time, csv
from itertools import combinations

def exhaustive_airlift(w, v, cap):
    n = len(w)
    best_val, best_mask = -1, 0
    examined = feasible = 0
    for mask in range(1 << n):                  # every bit-string = one candidate subset
        examined += 1
        tw = tv = 0
        for i in range(n):
            if (mask >> i) & 1:                 # bit i = 1  -> package i is loaded
                tw += w[i]; tv += v[i]
        if tw <= cap:                           # feasibility check
            feasible += 1
            if tv > best_val:                   # keep the best feasible subset
                best_val, best_mask = tv, mask
    return best_val, best_mask, examined, feasible

def mask_to_ids(mask, n):
    return [f"P{i+1}" for i in range(n) if (mask >> i) & 1]

def verify_small(w, v,
        cap):                   # independent checks for the small instance
    n = len(w)
    best = max((sum(v[i] for i in c), c) for r in range(n + 1)
               for c in combinations(range(n), r) if sum(w[i] for i in c) <= cap)
    dp = [0] * (cap + 1)                        # dynamic-programming check
    for wi, vi in zip(w, v):
        for c in range(cap, wi - 1, -1):
            dp[c] = max(dp[c], dp[c - wi] + vi)
    return best[0], [f"P{i+1}" for i in best[1]], dp[cap]

if __name__ == "__main__":
    w, v, cap = [12, 7, 11, 8, 9], [24, 13, 23, 15, 16], 26
    print("Small instance: weights", w, " values", v, " payload limit", cap)
    print(f"{'Subset':22}{'Weight':>8}{'Value':>7}  Feasible")
    for mask in range(1 << 5):
        tw = sum(w[i] for i in range(5) if mask >> i & 1)
        tv = sum(v[i] for i in range(5) if mask >> i & 1)
        print(f"{'{' + ','.join(mask_to_ids(mask, 5)) + '}':22}{tw:>8}{tv:>7}  "
              f"{'Yes' if tw <= cap else 'No'}")
    bv, bm, ex, fe = exhaustive_airlift(w, v, cap)
    print(f"\nExhaustive search : value={bv}, packages={mask_to_ids(bm, 5)}, "
          f"examined={ex}, feasible={fe}")
```

Screenshot 16: Source code, q6_exhaustive.py (part 1 of 2)

```python
    print("Independent checks:", verify_small(w, v, cap))

    print(f"\n{'n':>4}{'2^n (predicted)':>17}{'examined':>12}{'feasible':>10}"
          f"{'time(s)':>10}{'pred. time(s)':>15}")
    rows, base = [], None
    for n in (10, 12, 14, 16, 18, 20):
        r = random.Random(n)
        w = [r.randint(5, 50) for _ in range(n)]
        v = [r.randint(10, 100) for _ in range(n)]
        cap = sum(w) // 2
        t0 = time.perf_counter()
        bv, bm, ex, fe = exhaustive_airlift(w, v, cap)
        t = time.perf_counter() - t0
        if base is None: base = (n, t)
        pred = base[1] * 2 ** (n - base[0]) * n / base[0]
        rows.append([n, 2 ** n, ex, fe, round(t, 4), round(pred, 4)])
        print(f"{n:>4}{2**n:>17,}{ex:>12,}{fe:>10,}{t:>10.3f}{pred:>15.3f}")
    c = sum(r_[4] / (r_[0] * 2 ** r_[0]) for r_ in rows[-3:]) / 3
    print(f"\nFitted model: t = {c:.3e} * n * 2^n")
    for limit in (1, 10, 60):
        n = 1
        while c * (n + 1) * 2 ** (n + 1) <= limit: n += 1
        print(f"Largest n within {limit:>2} s : n = {n}  (n+1 would need "
              f"{c*(n+1)*2**(n+1):.1f} s)")
    with open("data/q6_raw.csv", "w", newline="") as fh:
        w_ = csv.writer(fh)
        w_.writerow(["n","2^n","examined","feasible","time_s","predicted_time_s"])
        w_.writerows(rows)
```

Screenshot 17: Source code, q6_exhaustive.py (part 2 of 2)

6.2 Correctness check on a small instance

Instance: five packages with weights 12, 7, 11, 8, 9, humanitarian values 24, 13, 23, 15, 16, and a payload limit of 26. There are $2^5 = 32$ subsets, so all of them can be checked by hand; the program prints the full table (see the output screenshot below). The best feasible subsets are:

Table 14: Best feasible combinations (hand-checkable)

Rank	Packages	Total weight	Total value
1	{P2, P3, P4}	7 + 11 + 8 = 26	13 + 23 + 15 = 51
2	{P1, P3}	12 + 11 = 23	24 + 23 = 47
3	{P2, P4, P5}	7 + 8 + 9 = 24	13 + 15 + 16 = 44
4	{P1, P5}	12 + 9 = 21	24 + 16 = 40

Of the 32 subsets, 18 are feasible. The algorithm returned **value 51 with packages P2, P3 and P4**, which uses exactly the 26-unit limit. Two independent methods agree: a check using Python's itertools.combinations and a dynamic-programming table both give 51. The algorithm therefore finds the correct optimum.

```
Terminal
$ python q6_exhaustive.py
Small instance: weights [12, 7, 11, 8, 9]  values [24, 13, 23, 15, 16]  payload limit 26
Subset                  Weight  Value  Feasible
{}                           0      0  Yes
{P1}                        12     24  Yes
{P2}                         7     13  Yes
{P1,P2}                     19     37  Yes
{P3}                        11     23  Yes
{P1,P3}                     23     47  Yes
{P2,P3}                     18     36  Yes
{P1,P2,P3}                  30     60  No
{P4}                         8     15  Yes
{P1,P4}                     20     39  Yes
{P2,P4}                     15     28  Yes
{P1,P2,P4}                  27     52  No
{P3,P4}                     19     38  Yes
{P1,P3,P4}                  31     62  No
{P2,P3,P4}                  26     51  Yes
{P1,P2,P3,P4}               38     75  No
{P5}                         9     16  Yes
{P1,P5}                     21     40  Yes
{P2,P5}                     16     29  Yes
{P1,P2,P5}                  28     53  No
{P3,P5}                     20     39  Yes
{P1,P3,P5}                  32     63  No
{P2,P3,P5}                  27     52  No
{P1,P2,P3,P5}               39     76  No
{P4,P5}                     17     31  Yes
{P1,P4,P5}                  29     55  No
{P2,P4,P5}                  24     44  Yes
{P1,P2,P4,P5}               36     68  No
{P3,P4,P5}                  28     54  No
{P1,P3,P4,P5}               40     78  No
{P2,P3,P4,P5}               35     67  No
{P1,P2,P3,P4,P5}            47     91  No

Exhaustive search : value=51, packages=['P2', 'P3', 'P4'], examined=32, feasible=18
Independent checks: (51, ['P2', 'P3', 'P4'], 51)

   n  2^n (predicted)    examined  feasible   time(s)  pred. time(s)
  10            1,024       1,024       512     0.001         0.001
  12            4,096       4,096     2,048     0.003         0.003
  14           16,384      16,384     8,192     0.013         0.012
  16           65,536      65,536    32,768     0.055         0.057
  18          262,144     262,144   131,870     0.241         0.257
  20        1,048,576   1,048,576   524,288     1.056         1.141

Fitted model: t = 5.140e-08 * n * 2^n
Largest n within  1 s : n = 19  (n+1 would need 1.1 s)
Largest n within 10 s : n = 23  (n+1 would need 20.7 s)
Largest n within 60 s : n = 25  (n+1 would need 89.7 s)
$ _
```

Screenshot 18: Program output of q6_exhaustive.py

6.3 Growth investigation

Prediction before experiments

- Each package is either in or out of the load, so there are 2 choices per package and 2 × 2 × … × 2 = **2^n candidate subsets**.

- Adding one package **doubles** the candidates; adding 10 packages multiplies them by 2^{10} = 1,024.
- Each candidate costs O(n) to evaluate, so the running time should be proportional to n × 2^n. I predicted the time from the n = 10 measurement using $T(n) = T(10) \times 2^{n-10} \times n/10$.

Table 15: Subsets examined and time (random instances, capacity = half of the total weight)

n	2^n (predicted)	Subsets examined	Feasible	Time (s)	Predicted time (s)
10	1,024	1,024	512	0.0006	0.0006
12	4,096	4,096	2,048	0.0027	0.0027
14	16,384	16,384	8,192	0.0128	0.0125
16	65,536	65,536	32,768	0.0554	0.0571
18	262,144	262,144	131,870	0.2406	0.2568
20	1,048,576	1,048,576	524,288	1.0562	1.1414

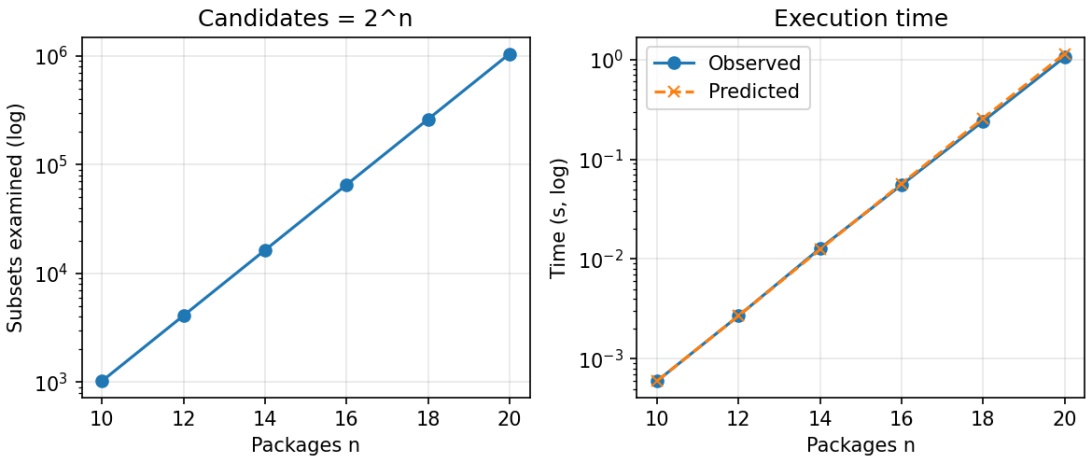


Figure 7: Candidates (2^n) and execution time against n (log scale)

Six problem sizes were used (n = 10 to 20). The number of subsets examined equals 2^n exactly in every case, and the predicted times are within about 8% of the measured times. Going from n = 10 to n = 20 multiplied the candidates by 1,024 and the time from 0.001 s to 1.06 s.

6.4 Inquiry: why does a small increase in packages cause a dramatic increase in effort?

The solution space is not a list of n items but the set of all **combinations** of them, whose size is 2^n. Every new package is an independent yes/no decision which doubles the number of existing combinations: each old subset now has two versions, one without the new package

and one with it. Growth therefore compounds multiplicatively, whereas adding an item to a linear search adds only one more step.

Using the fitted model $t \approx 5.14 \times 10^{-8} \times n \times 2^n$ seconds, ten extra packages beyond $n = 20$ would not cost 10 times more but about 1,500 times more:

Table 16: Extrapolated running time of the exhaustive search

n	Subsets	Estimated time
20	1,048,576	about 1 second (measured)
30	1,073,741,824	about 1,656 s (28 minutes)
40	1.1×10^{12}	about 2.3×10^{6} s (26 days)
50	1.1×10^{15}	about 2.9×10^{9} s (about 92 years)

6.5 Deployment decision

The airlift planner must produce a loading decision, but not in real time: a plan within **10 seconds** is acceptable for briefing the crew, and this is the response time I selected. Using the fitted model:

Table 17: Estimated time near the limit

n	Estimated time	Decision
21	2.3 s	Comfortable
22	4.7 s	Deploy with a safety margin of 2×
23	9.9 s	At the limit, no margin on slower hardware
24	20.7 s	Rejected

Deployment boundary: 22 packages

I would deploy the exhaustive-search solution for **up to 22 packages** (estimated 4.7 s), which leaves about a factor of two to absorb slower hardware or longer package lists in the worst case. At 23 packages the estimate reaches the 10 s limit with no margin, and one more package doubles it again.

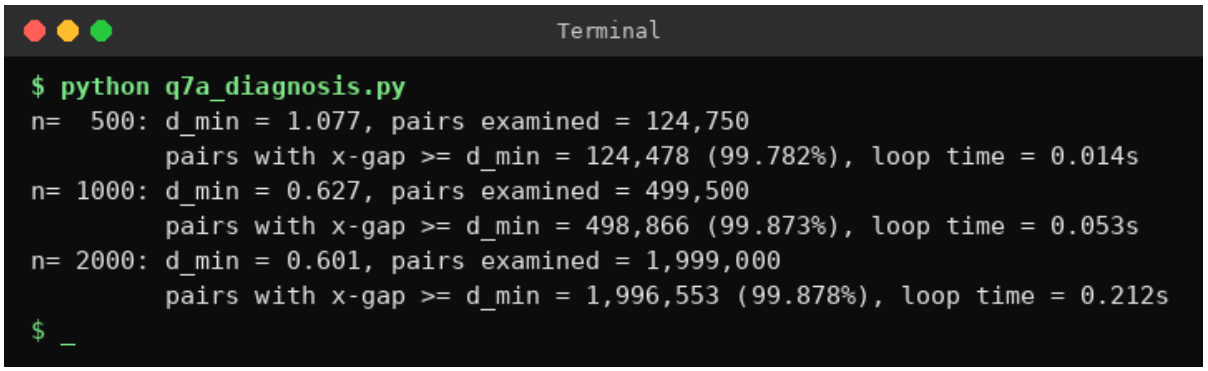
Beyond that size the planner should switch to an approach that does not examine every subset, such as pruning candidates that already exceed the payload or dynamic programming.

Q7 – Diagnose, Redesign and Validate (13 marks)

Selected solution: Q4, the brute-force closest pair for drones. Its Q4 evidence shows the clearest limitation: the work grows as n^2 and became impractical at about 4,000 drones in my 1-second criterion.

A. Diagnose the computational bottleneck (2 marks)

- **Where the effort is spent:** almost entirely in the body of the inner loop, which computes dx, dy and the squared distance for each pair. In Q4 the cost was a steady 105 ns per pair, and the algorithm has no other significant cost.
- **Which operation grows:** the distance calculation. Its count is exactly $n(n-1)/2$: 499,500 for n = 1,000 and 1,999,000 for n = 2,000 (4× for 2× the drones), and 17,997,000 for n = 6,000.
- **Is work unnecessarily repeated or wasted?** Yes. To measure it, I counted for the same random data how many pairs have an x-gap $|x_i - x_j|$ already **larger than the final answer (d_min)**. Such a pair cannot be the closest pair, because the straight-line distance is never smaller than the x-gap (Pythagoras: $\sqrt{(dx^2 + dy^2)} \geq |dx|$). The result:



Screenshot 19: Program output of q7a_diagnosis.py

n	Closest distance d_min	Pairs examined	Pairs with x-gap ≥ d_min	Wasted share
500	1.077	124,750	124,478	99.78%
1,000	0.627	499,500	498,866	99.87%
2,000	0.601	1,999,000	1,996,553	99.88%

- **Which characteristic of the algorithm causes this:** it is order-blind and bound-blind. It treats the positions as an unordered set, so it cannot know which drones are near each other without computing, and it never uses the best distance found so far to reject a pair without computing it. More than 99.7% of its distance calculations in these runs were for pairs that could be dismissed by one subtraction.

B. Design objective (2 marks)

The redesign must meet the following measurable objectives while returning **exactly the same closest pair distance**:

1. Reduce distance calculations by **at least 95%** for n = 2,000 uniform random drones (from 1,999,000 to at most about 100,000).
2. Change the growth of the distance-calculation count from quadratic to clearly **sub-quadratic**: when n doubles, the count should grow by clearly less than 4×.
3. Keep the result exact (no approximation) and keep the extra memory to O(n).

C. Revised computational strategy (4 marks)

What changes and what property is exploited

Property: for any two points, the Euclidean distance is at least the gap along one coordinate axis. If the points are **sorted along that axis**, then once the gap from point i to point j is already at least the best distance found so far, every point after j has an even larger gap, so none of them can improve the answer. The whole remainder of the inner loop can be skipped, not just a single pair.

The redesign uses this in three steps: (1) sort the drones once along one axis; (2) for each drone, scan forward only while the axis gap is smaller than the current best distance; (3) choose the axis with the larger spread of coordinates, so the gaps are informative.

```
ALGORITHM PrunedClosestPair(D[0..n-1])
  axis ← coordinate (x or y) with the larger range of values
  S ← D sorted by that coordinate            // preprocessing
  best ← ∞
  for i ← 0 to n-2:
    for j ← i+1 to n-1:
      gap ← S[j].axis - S[i].axis
      if gap^2 ≥ best: break     // later points are even farther
      d2 ← gap^2 + (other-coordinate difference)^2
      if d2 < best: best ← d2
  return sqrt(best)
```

Table 18: Summary of the design changes

Question	Answer

Question	Answer
What was changed?	The inner loop is no longer run to the end; it stops as soon as the gap along the sort axis reaches the best distance found so far.
What is exploited?	Ordering (sorted coordinates) and a lower bound (axis gap ≤ distance), together with the best distance found so far.
What is avoided?	Distance calculations (and loop iterations) for pairs that cannot be the closest.
Preprocessing?	Yes: one sort, O(n log n) time.
Extra memory?	One sorted copy of the points, O(n).
New trade-offs?	Sorting cost; code is more complex; the benefit depends on how the data is spread (see F); worst case still $O(n^2)$ if many points share nearly the same coordinate on the chosen axis.

```python
# Q7 - Redesign of Q4: sort once, then prune candidate pairs using the x-gap lower bound
import random, time, csv
from q4_closest_pair import closest_pair_bf

def closest_pair_pruned(pts, axis="auto"):
    """Sort along one axis; for each point only scan forward while the gap along
    that axis is still smaller than the best distance found so far."""
    if axis == "auto":                                  # use the axis with larger spread
        xs = [p[0] for p in pts]; ys = [p[1] for p in pts]
        axis = 0 if max(xs) - min(xs) >= max(ys) - min(ys) else 1
    other = 1 - axis
    s = sorted(pts, key=lambda p: p[axis])              # preprocessing: O(n log n)
    n = len(s)
    best, dists, gap_checks = float("inf"), 0, 0
    for i in range(n - 1):
        for j in range(i + 1, n):
            gap = s[j][axis] - s[i][axis]
            gap_checks += 1
            if gap * gap >= best:                       # every later point is even farther
                break
            dy = s[j][other] - s[i][other]
            d2 = gap * gap + dy * dy
            dists += 1
            if d2 < best:
                best = d2
    return best ** 0.5, dists, gap_checks

def uniform(n, seed=5):
    r = random.Random(seed); return [(r.uniform(0, 1000), r.uniform(0,
            1000)) for _ in range(n)]
def clustered(n, seed=6):
    r = random.Random(seed); cs = [(200, 200), (800, 300), (500, 850)]
    return [(r.gauss(cs[k % 3][0], 40), r.gauss(cs[k % 3][1],
            40)) for k in range(n)]
def vertical_line(n, seed=7):
    r = random.Random(seed); return [(500.0, r.uniform(0, 1000)) for _ in range(n)]

if __name__ == "__main__":
    rows = []
    print("Original (brute force) vs redesigned (sort + prune) - uniform random drones")
    print(f"{'n':>7}{'orig dist calcs':>17}{'new dist calcs':>16}{'reduction':>11}"
          f"{'orig t(s)':>11}{'new t(s)':>10}  same answer")
    for n in (500, 1000, 2000, 4000, 8000, 16000, 32000, 64000):
        pts = uniform(n)
        if n <= 4000:
            t0 = time.perf_counter()
```

Screenshot 20: Source code, q7_redesign.py (part 1 of 2)

```
            d0, _, _, oc = closest_pair_bf(pts)
            ot = time.perf_counter() - t0
        else:
            oc, ot, d0 = n * (n - 1) // 2, None, None     # original count is exactly n(n-1)/2
        t0 = time.perf_counter()
        d1, nc, gc = closest_pair_pruned(pts)
        nt = time.perf_counter() - t0
        same = "yes" if d0 is None or abs(d0 - d1) < 1e-9 else "NO"
        rows.append([n, oc, nc, gc, ot if ot is None else round(ot, 4), round(nt,
                4)])
        ots = f"{ot:.3f}" if ot is not None else "(not run)"
        print(f"{n:>7,}{oc:>17,}{nc:>16,}{100*(1-nc/oc):>10.2f}%{ots:>11}{nt:>10.3f}  {same}")
    print("\nOther data distributions, n = 2000")
    print(f"{'distribution':22}{'orig calcs':>12}{'x-axis only':>13}{'auto axis':>11}")
    drows = []
    for name, f in [("uniform", uniform), ("clustered (3 groups)", clustered),
            ("all drones on x=500", vertical_line)]:
        pts = f(2000)
        d0, _, _, oc = closest_pair_bf(pts)
        d1, c1, _ = closest_pair_pruned(pts, axis=0)
        d2, c2, _ = closest_pair_pruned(pts, axis="auto")
        assert abs(d0 - d1) < 1e-9 and abs(d0 - d2) < 1e-9
        drows.append([name, oc, c1, c2])
        print(f"{name:22}{oc:>12,}{c1:>13,}{c2:>11,}")
    with open("data/q7_raw.csv", "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["n","orig_dist_calcs","new_dist_calcs","new_gap_checks","orig_time_s","new_time_s"])
        w.writerows(rows)
    with open("data/q7_distributions.csv", "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["distribution","orig","x_axis_only","auto_axis"])
        w.writerows(drows)
```

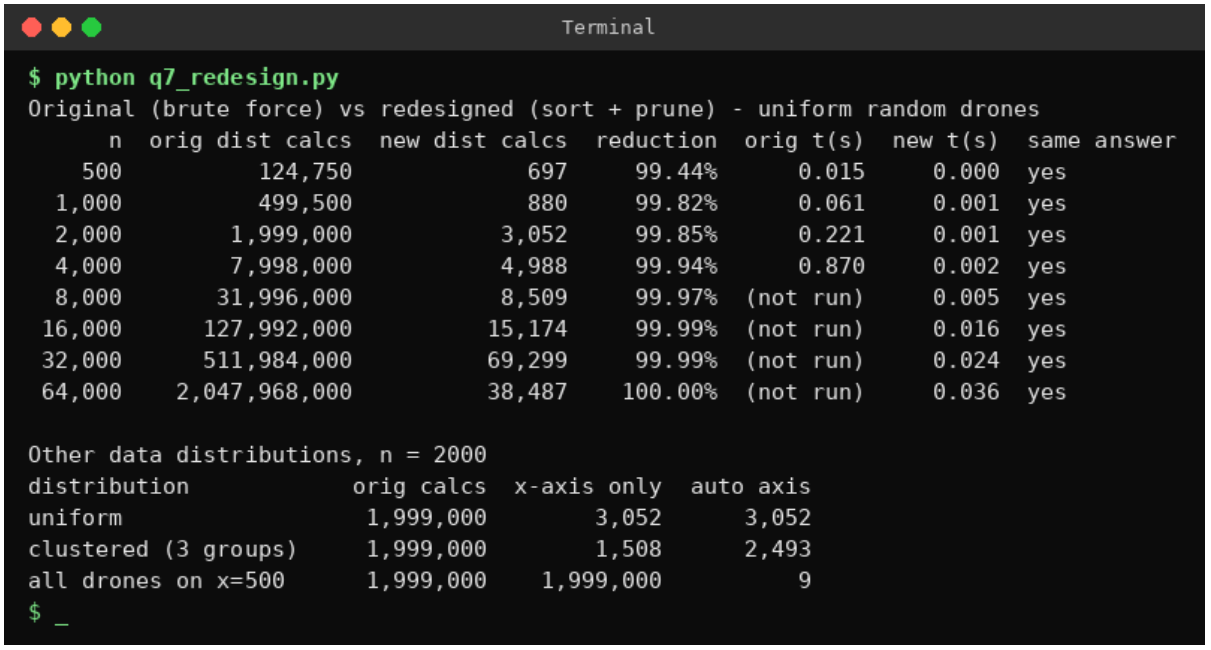
Screenshot 21: Source code, q7_redesign.py (part 2 of 2)

D. Prediction before implementing (1 mark)

- **Measure expected to improve:** the number of distance calculations. I expect a reduction of more than 99% for n = 2,000 (the diagnosis shows 99.88% of pairs are wasted), which is above the 95% objective.
- **Growth:** close to linear in n (roughly 2× for each doubling of n) for uniformly spread drones, since once a small best distance is found only a few neighbours lie within it along the sorted axis.
- **Weakness predicted:** if all drones have the same x coordinate (gap always 0), pruning on x can never fire, so a fixed x-axis version should be no better than the original; the choice of the better axis should fix this.
- **Execution time:** should fall too, even including the sort, but this is supporting evidence only.

E. Implementation and validation (2 marks)

The same random drone sets as Q4 (seed 5) were used. The original function is run up to n = 4,000; for larger n its count is exactly n(n−1)/2, a result proved and confirmed in Q4. The redesigned distance is compared with the original for every n up to 4,000 and for all three distributions below, and the answers were identical in all of these.



Screenshot 22: Program output of q7_redesign.py

Table 19: Original against redesign (uniform random drones)

n	Original distance calcs	Redesign distance calcs	Redesign gap checks	Reduction	Original time (s)	Redesign time (s)
500	124,750	697	1,196	99.44%	0.0148	0.0003
1,000	499,500	880	1,879	99.82%	0.061	0.0005
2,000	1,999,000	3,052	5,049	99.85%	0.2208	0.0011
4,000	7,998,000	4,988	8,987	99.94%	0.8695	0.0023
8,000	31,996,000	8,509	16,507	99.97%	not run	0.0046
16,000	127,992,000	15,174	31,173	99.99%	not run	0.0161
32,000	511,984,000	69,299	101,298	99.99%	not run	0.0242
64,000	2,047,968,000	38,487	102,486	100.00%	not run	0.0358

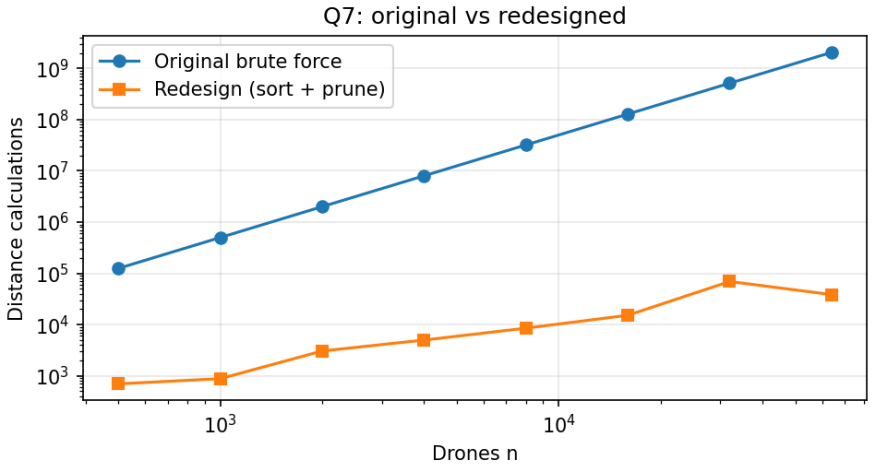


Figure 8: Distance calculations: original against redesign (log–log)

Table 20: Other data distributions (n = 2,000)

Distribution	Original	Redesign, x-axis only	Redesign, automatic axis
uniform	1,999,000	3,052	3,052
clustered (3 groups)	1,999,000	1,508	2,493
all drones on x=500	1,999,000	1,999,000	9

F. Defend the final design (2 marks)

- **Was the prediction supported?** Yes. At n = 2,000 distance calculations fell from 1,999,000 to 3,052 (99.85% fewer, against a 95% objective). At n = 4,000 the original took 0.87 s and the redesign 0.002 s.
- **Magnitude of improvement:** between 99.4% and almost 100% fewer distance calculations for n = 500 to 64,000. Across a 128-fold increase in n (500 to 64,000), the redesign's calculations grew only about 55 times (697 to 38,487), roughly linear, compared with the 16,400-fold growth of the original. At 64,000 drones the original would need about 2 billion distance calculations; the redesign needs about 38 thousand.
- **Observed irregularity:** the growth is not perfectly smooth (for example 69,299 calculations at n = 32,000 against 38,487 at n = 64,000). The count depends on how early a small best distance is found, which depends on the random layout, not only on n. This does not affect correctness.
- **New costs:** the O(n log n) sort, an extra O(n) copy of the data, about as many gap checks as there are points, and more complicated code. The gap checks are cheap subtractions, so they do not remove the advantage.

- **Weakness confirmed by the experiment:** with all drones on one vertical line (x = 500), the x-axis-only version still examined 1,999,000 pairs, which is the same as the original and in fact slightly worse because of the sorting overhead. Choosing the axis with the larger spread fixed it: only 9 calculations. For clustered data the fixed axis was actually a little better than automatic selection (1,508 against 2,493), so the spread rule is a good heuristic but not a guarantee.
- **When the original may still be preferable:** for very small n (a few dozen drones) where the sort and the extra code bring no measurable gain; when the code must be as simple as possible to verify; and as a reference solution to test the faster one, which is how I used it in Q7.

Final conclusion

The redesign **achieved its objective**: more than 99% fewer distance calculations, close to linear growth, and an identical result, using only one extra property (axis gap ≤ distance) plus a sort.

It does **not** remove the quadratic worst case: an adversarial layout where many drones share almost the same coordinate on the chosen axis would still force many pair examinations. A guaranteed O(n log n) bound would need a divide-and-conquer strategy, which was outside the scope of this question.